College of Computer Studies

Computer Science

Machine Learning-Based IT Internship Matching and Career Recommendation System for Computer Science and Information Technology Students

Finals

Cabalagnan, Lawrenz
Caña, Raiden Clark A.
Javier, Kassidy Glean
Pangan, Gion Jickain
Uy, Sem

Submitted to:

Mr. Francis Carabuena

Table Of Contents

Title Page..... 1

Chapter 1: Introduction..... 4

Background of the Study..... 4

Problem Statement..... 5

Objectives..... 5

General Objective..... 5

Specific Objectives..... 6

Significance of the Study..... 6

Scope and Delimitation..... 7

Scope..... 7

Delimitation..... 7

Chapter 2: Related Literature and Studies..... 8

Related Literature..... 8

Resume Parsing and Information Extraction..... 8

Text Classification for Skill Proficiency..... 9

Content-Based Recommendation..... 10

Related Studies..... 10

Resume Parsing and Screening..... 10

Resume–Job Matching Models..... 11

Reviews of Recruitment and Internship Recommender Systems..... 11

Fairness and Validity in Algorithmic Hiring..... 12

Synthesis and Research Gap..... 12

Chapter 3: Methodology (Part 1)..... 16

Research Design..... 16

SDLC Model....16
Chosen Model....16
Why This Model Fits the Project....16
Phases....16
Diagram of the Model....17
Functional Requirements....18
Non-Functional Requirements....19
Performance....19
Security....19
Usability....19
Compatibility....19
Algorithm Formula....19
A. Resume Skill Extraction....19
B. Proficiency Estimation....20
C. Internship Match Score....20
System Presentation (Step by Step Documentation)....20
References....22
Appendices....23
Appendix A: Screenshots....23
Appendix B: Evidence for the Match Percentage....29
Appendix C: Training Data Disclosure....29

Chapter 1: Introduction

Background of the Study

Finding an internship is an important activity for Computer Science and Information Technology students because it allows them to apply knowledge in a professional environment. However, this task is not always easy: students have to analyze numerous postings to find out which fits their current skillset and experience level. At the same time, companies trying to fill a position have to analyze applicants and their resumes, comparing their skills to the desired ones.

ITern is a web application meant to help both sides with finding a proper match. Its user-friendly interface allows students to create a profile, upload a PDF resume, see extracted skills, confirm the known ones using skill tests, find and apply to internships matching their skill set and track the progress of their applications. The companies can post their available internships, define the set of required skills, review the applicants and track the progress of their applications.

The application uses two different processes worth noting. Firstly, the skill extraction from resumes uses a predefined list of skill names and aliases, which are matched to the text of the resume. Secondly, a trained text-classification model is used to predict the skill proficiency based on surrounding words. Finally, the calculation of a percentage match between a student profile and an internship posting is made by comparing the stored skill levels to the required ones with additional weight multipliers. Thus, it should be noted, that the match percentage is a calculated value based on the available information and skills, but is not an indicator of a student’s chances at succeeding at the internships

Problem Statement

Students may have a hard time comparing internship requirements with the skills and experience listed in their resume. Companies, on the other hand, would need an efficient way of assessing candidates in accordance with specific requirements for every internship. ITern tackles every issue, including organization of student profiles and experience, extraction of relevant skills from the resumes, calculation of compatibility scores, and application management.

Specifically, the study seeks to answer the following questions:

1. How can a web-based system organize student and company accounts, profiles, and internship postings?
2. How can ITern identify recognized technical skills from the text extracted from a student's PDF resume?
3. How can a text-classification model estimate a proficiency category from wording around a detected skill?
4. How can the system calculate and display a match percentage between a student's stored skills and an internship's required skills?
5. How can the system support application submission, capacity-based waitlisting, applicant review, and interview scheduling?
6. How do intended users evaluate the system's functionality, usability, and usefulness? (Complete this question only after conducting and documenting an actual evaluation.)

Objectives

General Objective

To design and develop ITern, a web-based system to match Computer Science and Information Technology students and companies for internships and careers by utilizing resume skill extraction, proficiency estimation, and skill based match scoring to aid in finding internships as well as reviewing applicants.

Specific Objectives

1. To develop separate student and company account flows with email OTP verification.
2. To provide student profiles for educational information, career goals, skills, and PDF resumes.
3. To extract recognized skills from resume text using a controlled vocabulary and defined aliases.
4. To estimate skill proficiency categories using a trained TF-IDF and logistic-regression text classifier.
5. To calculate internship compatibility scores from required-skill coverage and matched-skill strength.
6. To allow students to browse internships, apply, track application status, and take skill-verification tests.
7. To allow companies to manage postings, review applicants, view match scores, update application statuses, and schedule interviews.
8. To test the implemented features and document the results without claiming performance that has not been measured.

Significance of the Study

- Students: ITern offers one place to organize skills and resumes, discover relevant internship postings, and track applications. Match scores can help students compare postings, but should not be treated as a guarantee of acceptance or job performance.
- Companies: The system organizes internship postings and applicant information and provides a consistent score based on the skills recorded in the system.
- The institution: The project demonstrates the application of web development, database design, resume text processing, and machine-learning classification in a student-developed system.

- Future researchers and developers: The documentation can serve as a starting point for improving skill extraction, evaluating matching methods, and replacing synthetic training examples with appropriately sourced and consented real-world data.

Scope and Delimitation

Scope

- Web-based system with separate student and company portals.
- Student account registration and email OTP verification, student profile management, career goal, skill records, and PDF resume upload.
- Resume text extraction from PDF files and detection of skills from a fixed, code-defined vocabulary and aliases.
- Proficiency classification into Beginner, Intermediate, or Advanced categories using a model trained on labeled example sentences.
- Skill-based internship matching using required-skill coverage and skill-strength calculations, with required-skill weights taken from the posting.
- Skill verification tests, internship browsing and applications, capacity-based waitlisting, company applicant review, application-status updates, interview scheduling, and notifications, as implemented in the supplied project.
- Local development configuration using HTML, CSS, JavaScript, Node.js/Express, MySQL, and Python/FastAPI.

Delimitation

The current version is a prototype for academic purposes, not a production hiring or applicant tracking system. The resume skill detector only detects skills that are in the vocabulary and aliases of the skill detector, and not necessarily skills that are every possible skill or wording. The extraction from PDFs works if the PDFs have selectable text, but otherwise (e.g., scanned) may not be parsed properly by OCR. The proficiency model was trained on synthetic sentences created from wording templates because the project did not have access to a verified set of resumes with verified proficiency labels. Therefore, the model's predicted proficiency is a

prediction about the pattern of wording, rather than a measurement of the skill of the person with the resume. This system does not check the authenticity or validity of the claims on a resume, certificates, employment history, or skills. The match percentages are compatibility scores based on rules, and are not an estimated probability of being hired. The test accuracy of the model is only on the held-out synthetic examples, and should not be interpreted as an accuracy claim on real resumes. This system only has the functionality as implemented in the supplied code, and does not guarantee internships.

Chapter 2: Related Literature and Studies

Related Literature

Resume Parsing and Information Extraction

Information extraction refers to the task of converting unstructured text into structured records that a software can store and query (Sarawagi, 2008). Resume parsing is an application of this task to the documents of applicants: text is extracted from the file and mapped into fields, such as education, experience, and skills. The approaches range from hand-written rules and dictionary lookup to statistical models trained on annotated text (Sarawagi, 2008).

ITern uses the simplest and most transparent of these approaches, the controlled vocabulary. After text is extracted from a PDF resume and converted into lowercase, each of the configured skill aliases is searched using word-boundary-aware matching and mapped to a canonical skill name. For example, “JS” is mapped to JavaScript and “React.js” or “ReactJS” are mapped to React. Because every detection is traceable to a specific word in the resume, the result is simple to explain and easy to verify. The trade-off is in coverage: skills that are not present in the vocabulary or that are written with unrecognized phrases are missed, and scanned resumes with no selectable text cannot be read without optical character recognition. A detected skill therefore only shows that a term appears in the resume but not that the student is proficient in it.

Text Classification for Skill Proficiency

Text classification assigns one of several predefined categories to a piece of text by using a model learned from labeled examples (Sebastiani, 2002). A common pipeline first transforms the text into a set of numerical features and then learns a classifier over these features. TF-IDF is a popular way to weigh terms: its value is given by how often the term occurs in the text multiplied by how rare the term is across all texts, so that terms that well-discriminate examples get higher weights (Salton & Buckley, 1988; Spärck Jones, 1972). Robertson (2004) provides an overview of theoretical arguments for this choice of inverse document frequency. Multinomial logistic regression is a standard classifier for such feature sets: it learns a weight for every feature and every class, and interprets the weighted sum of features as class probabilities (Hosmer et al., 2013). All components of this pipeline are implemented in scikit-learn (Pedregosa et al., 2011).

ITern follows this pipeline to classify the wording around a detected skill into Beginner, Intermediate, and Advanced. The predicted label is then converted into an internal numeric level (35, 65, or 88). The classifier is only used to estimate proficiency: it does not select internships by itself.

The main limitation is that the examples the classifier was trained on were generated synthetically. Supervised classifiers cannot be better than the examples they learn from (Sebastiani, 2002), and ITern's training script only combines 10 proficiency-related sentence templates with 18 skill names for each of 3 classes, which gives 540 labeled sentences in total. They are split into a stratified 80/20 train/test set with a fixed random seed to give 432 training and 108 test examples. While this design makes the training pipeline deterministic, sentences generated from the same templates will have the same patterns in both the training and the test sets, and thus a high score on the test set is not indicative of the classifier's ability to generalize to the diverse wording patterns of real resumes labeled by qualified evaluators. Any accuracy claim about the model in this post should be considered to refer to its performance on the synthetic test set only.

Content-Based Recommendation

A content-based recommender compares the attributes of items in a collection with a profile of the user and ranks the items based on degree of match (Lops et al., 2011; Ricci et al., 2015). In ITern, the student's recorded skills constitute the profile while the required skills in each internship posting represent item attributes. One advantage of a content-based approach is explainability: since the recommendation is based on easily interpretable attributes, the rationale for a high-ranked item is apparent to the user (Lops et al., 2011).

ITern uses a rule-based score that takes into account both coverage of required skills and the relative strength of each matched skill, weighted according to skill proficiencies stated in the job post. The 60/40 split between coverage and strength, the numeric values corresponding to each proficiency label, and the use of a fixed score of 12 for job postings with required skills but no matches are design choices made by the project team. These are not values that were found through literature review or that were fit to data about hiring outcomes. The score does not account for other factors that may contribute to a student's suitability for an internship, such as interview performance, communication proficiency, work authorization, availability, or accuracy of information on a resume.

Related Studies

Resume Parsing and Screening

Bhatia et al. (2019) developed a resume screening system as an end-to-end solution consisting of two steps. In the first stage, a resume parser extracts relevant information in a structured manner. Then, a BERT sentence-pair model ranks candidates according to their match to a given job description, based on the candidates' experience in their previous positions approximated as a distribution over job titles. The resume parser achieved 100% accuracy on the LinkedIn resume format, while the overall candidate screening accuracy reached 73%. Their solution demonstrates that the parsing and ranking stages can be combined in a single step, but the accuracy depends on the resume format and relies on training the language model.

Deshmukh and Raut (2025) utilize the BERT model for resume screening as well. The researchers requested 200 resumes with the participants' consent and used ten job descriptions retrieved from Glassdoor. The solution extracts relevant keywords and skill sets from the candidate's resume and calculates a similarity score compared to a job description, thus shortlisting candidates. According to the authors, their solution can screen up to one resume per second. Interestingly, their approach also starts with extracting relevant keywords and skills from the resume, which is similar to the initial step of ITern. However, instead of calculating the required skill coverage, their solution ranks candidates according to the text similarity.

Resume–Job Matching Models

Lavi et al. (2021) fine-tuned a Siamese Sentence-BERT model, conSultantBERT, on more than two hundred seventy thousand resume–vacancy pairs annotated by staffing consultants. Their results suggest that matching resumes to vacancies based only on shared words is not trivial: TF-IDF cosine similarity achieved an ROC-AUC score of around 0.57, while the best fine-tuned model scored around 0.85 on the same task. Resumes and vacancies describe complementary information; there is no overlap in the lexicon that would allow for simple keyword matching. This result justifies ITern's focus on canonical skill names rather than free-text descriptions and highlights its limitations: ITern is not able to detect semantically equivalent skills in different representations.

Zhu et al. (2018) described a person-job fit method that learns joint resume-job posting representations with convolutional networks, thus capturing the compatibility logic in the weights of these networks. Qin et al. (2018) built upon this work by developing an ability-aware neural network that decomposes job postings into abilities and resumes into experiences and focuses attention on the relevant abilities for a given job. Both works train their networks on large sets of historical recruitment records and operate with representations that are not easily interpretable to humans. ITern adopts the ability-weighted approach of Qin et al. (2018) but implements it as a simple formula with company-assigned weights.

Reviews of Recruitment and Internship Recommender Systems

De Ruijt and Bhulai (2021) conducted a systematic literature review of job recommender systems, focusing on articles published between 2011 and 2021. The review was based on content-based, collaborative, knowledge-based, and hybrid recommender systems. The authors concluded that fairness is rarely considered, and explainability is overlooked. Freire and de Castro (2021) performed a similar study of e-recruitment recommender systems, based on articles published between 2012 and 2020. Content-based recommendation was the most prevalent single type (26.98%), and the most common information source was resumes and job postings (42.85%). Overall, both reviews demonstrated that matching resume content to job requirements is a well-established design, while explainability and fairness are under-researched areas.

Poncio's (2024) literature review focused on internship profile matching. It included an inventory of the ways in which data are gathered to assess a profile, clustering techniques used to group similar profiles, and evaluation metrics used in such studies. The review identified gaps in the literature on internship recommender systems, particularly with regard to Southeast Asian students, and highlighted the value of such systems for university internship coordinators. This study directly supports the development of ITern by addressing internship profile matching in the context of students in the Philippines.

Fairness and Validity in Algorithmic Hiring

Kumar et al. (2023) surveyed fairness in recruitment recommender systems. They considered definitions of fairness, such as demographic parity and equal opportunity, methods to improve fairness, and how such findings relate to EU and US employment law, and warned that unfair outcomes in hiring systems may have legal repercussions. Raghavan et al. (2020) analyzed how vendors of algorithmic pre-employment assessments describe their mitigation efforts and validation methods, and emphasized the need for transparency (see also Bogen & Rieke, 2018). For this reason, ITern provides its match percentage as a compatibility score to facilitate comparisons between postings, but does not predict hiring success and the decision to hire a particular applicant remains with the company.

Synthesis and Research Gap

Table 2.1 summarizes the ten studies and indicates how ITern responds to each. Four issues emerge. First, studies of resume parsing and matching increasingly use BERT-based or other neural models (Bhatia et al., 2019; Deshmukh & Raut, 2025; Lavi et al., 2021; Qin et al., 2018; Zhu et al., 2018). Such methods can capture meaning beyond exact words, but require trained models and, in many cases, large labeled data sets, and are less interpretable. Second, the work of Lavi et al. (2021) highlights that word overlap is a weak signal for matching free text, which is why ITern matches canonical skill names rather than raw text. Third, reviews reveal that content-based matching of resumes and job posts is common, that explainability and fairness are often overlooked, and that internship-specific systems, particularly in Southeast Asia, are understudied (de Ruijt & Bhulai, 2021; Freire & de Castro, 2021; Poncio, 2024). Fourth, research on algorithmic hiring calls for transparency and caution about validity claims (Kumar et al., 2023; Raghavan et al., 2020).

Table 2.1 Synthesis of related studies and how ITern responds to them

Author/Year	Their Research Project	Findings	Our Solution (ITern)
Bhatia et al. (2019)	End-to-end resume parser and a BERT sentence-pair model that ranks candidates against a job description, released as a web application.	Parsed LinkedIn-format resumes with 100% accuracy and set a 73% accuracy baseline for candidate suitability. The setup was tested on resumes in only two format groups.	ITern extracts skills from PDF resumes with a transparent alias dictionary and scores matches with a rule-based formula, so no labeled resume–job pairs are needed. It also adds profiles, applications, and company-side management.
Deshmukh & Raut (2025)	BERT-based resume screening that extracts keywords and skill sets from 200 consented resumes and compares them with 10 job descriptions from Glassdoor.	Produced a similarity index for each resume to shortlist candidates; reported screening speed of up to 1 resume per second and similarity values reaching 0.3.	ITern also detects skills by keywords, but compares them with the posting's required skills and weights, shows the score to both students and companies, and lets students verify skills through tests.

Lavi, Medentsiy & Graus (2021)	conSultantBERT: a fine-tuned Siamese Sentence-BERT model that matches resumes and vacancies, trained on over 270,000 resume–vacancy pairs labeled by staffing consultants.	Fine-tuned embeddings clearly outperformed TF-IDF and plain BERT baselines (ROC-AUC about 0.85 vs. 0.57 for TF-IDF cosine similarity). Word overlap alone was a weak signal.	ITern avoids comparing free text. It maps resume wording to canonical skill names and compares them with the posting's required skills. This is more explainable, but it cannot recognize skills outside its vocabulary.
Zhu et al. (2018)	Person–job fit model that learns joint representations of resumes and job postings with convolutional networks, trained on real recruitment data.	Showed that learning representations of both documents together improves person–job fit prediction over baseline methods.	ITern uses no learned representation for matching. It uses an explicit formula of required-skill coverage (60%) and weighted skill strength (40%), which small teams can inspect, test, and defend without large datasets.
Qin et al. (2018)	Ability-aware neural network for person–job fit that splits postings into ability requirements and resumes into experiences, using attention to focus on the abilities that matter for a job.	Attention over job-specific abilities improved person–job fit prediction compared with baselines and made it clearer which requirements drive a match.	ITern gives each required skill a weight set by the company, so important skills count more in the strength component of the score and the reason for a score can be shown.
de Ruijt & Bhulai (2021)	Review of job recommender system literature from 2011 to 2021, including content-based, collaborative, knowledge-based, and hybrid systems.	Found many system types in use but noted that fairness is rarely considered and that explainability is often neglected.	ITern uses a content-based, explainable score built only from skills, levels, and posting weights, and states its limits openly. Fairness testing is left as future work.

Freire & de Castro (2021)	Systematic review of recommender systems in e-recruitment covering papers published from 2012 to 2020.	Content-based recommendation was the single most common named type (26.98%), and resumes and job posts were the most common information sources (42.85%).	ITern follows this common pattern, matching resume skills against job posts, but combines it with the full application workflow in a single system.
Poncio (2024)	Systematic review of job recommender techniques for internship profile matching, covering data collection, matching techniques, and evaluation measures.	Noted that reviews of internship recommender systems are limited, especially in Southeast Asia, and that such platforms can help university internship coordinators.	ITern is built specifically for Computer Science and Information Technology students and partner companies, with separate portals for each, as a prototype for a Philippine school setting.
Kumar et al. (2023)	Survey of fairness in recruitment recommender systems from technical and legal perspectives, covering fairness definitions and methods to reduce bias.	Warned that recommender systems in hiring can produce biased outcomes with legal consequences, and compared fairness methods against EU and US law.	ITern's score uses only skills, stored levels, and posting weights, and excludes personal attributes. The paper still treats the score as decision support only and does not claim it is bias-free.
Raghavan et al. (2020)	Study of how vendors of algorithmic pre-employment assessments describe their bias-mitigation and validation practices.	Highlighted the need for transparency about how bias is addressed and how the validity of hiring algorithms is established.	ITern documents its formula, its synthetic training data, and its limits, and states that match percentages are not probabilities of hiring success.

The gap which ITern aims to address is the absence of an integrated transparent workflow for managing internships that involve student profiles, resume-embedded skills extractions, skill-based matching, internship applications, and the management of applicants on the company side. What this study contributes is the implementation in one prototype academic system that embraces all of these elements. The paper does not go as far as to claim that either the proficiency model or the match score can be trusted to be effective in actual recruitment scenarios.

Chapter 3: Methodology (Part 1)

Research Design

The main design of this project is developmental, and its primary direction is aimed at creating, implementing, and testing new software products. Thus, this project's main products are the ITern web application and its resume-processing and matching services. The type of testing, which will be used for this project, is functional testing; it is applied to check whether each developed program's functions work correctly and meet the given requirements. Additionally, if a descriptive survey of users is conducted, the number of participants, the questions asked, the scale used, and the results received should be mentioned.

SDLC Model

Chosen Model

The project follows an iterative development approach. Features can be built and tested in manageable increments, including authentication, profiles, resume processing, matching, applications, and company-side management.

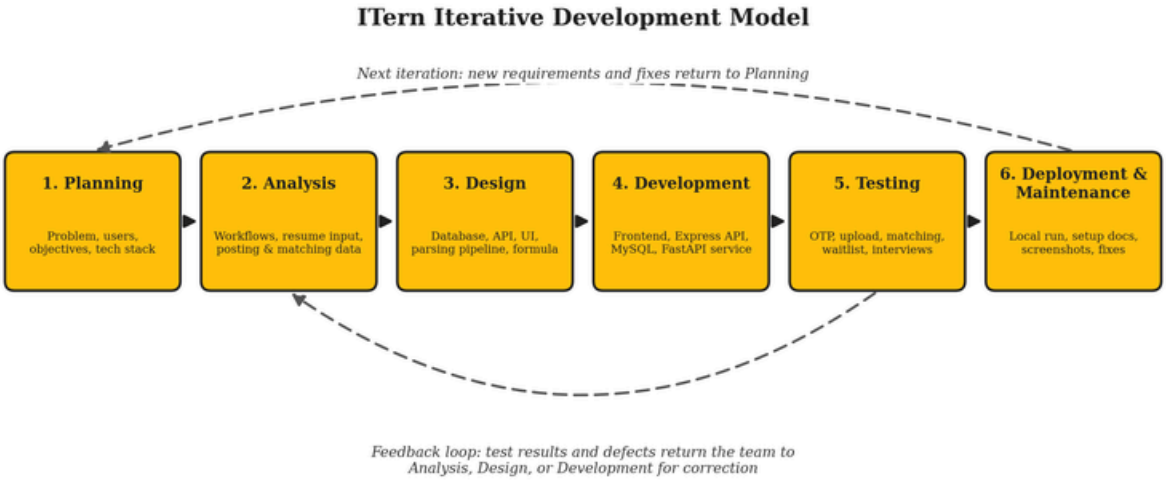
Why This Model Fits the Project

- The system has multiple connected modules, so implementing and checking them incrementally helps identify integration issues.
- The resume parsing and matching logic can be checked using sample inputs and known expected outputs while the interface is developed.
- Requirements and interface details may need revision after the team tests a feature.
- An iterative process lets the team fix defects and refine the workflow without waiting for every feature to be completed.

Phases

1. Planning: Define the problem, intended users, project objectives, initial feature list, technology stack, and work assignments.
2. Analysis: Identify student and company workflows, resume input requirements, posting requirements, application statuses, and data needed for matching.
3. Design: Design the database entities, API endpoints, student/company interfaces, resume-processing pipeline, and match-score formula.
4. Development: Implement the HTML/CSS/JavaScript frontend, Node.js/Express API, MySQL database interactions, and Python/FastAPI service.
5. Testing: Test registration and OTP flow, profile saving, PDF upload and text extraction, recognized-skill detection, proficiency predictions, match-score calculations, application capacity and waitlisting, status updates, and interview scheduling.
6. Deployment and Maintenance: Run the services in the configured local environment, document setup steps, capture screenshots, and fix issues found during testing.

Diagram of the Model



Functional Requirements

1. FR1: The system shall allow students and companies to register and sign in through separate role-specific flows.
2. FR2: The system shall verify registered email addresses using an OTP when email delivery is configured.
3. FR3: The system shall allow students to manage profile information, career goals, skills, and a PDF resume.
4. FR4: The system shall extract text from an uploaded PDF resume and identify skills that match the configured skill vocabulary and aliases.
5. FR5: The system shall estimate the proficiency category of a detected skill using the trained text classifier when the model file is available.
6. FR6: The system shall allow students to take available skill-verification tests and view their results.
7. FR7: The system shall allow companies to create, edit, activate, and close internship postings with required skills and weights.
8. FR8: The system shall calculate a compatibility score between a student's recorded skills and a posting's required skills.
9. FR9: The system shall display internship recommendations and match percentages to students.
10. FR10: The system shall allow students to submit applications and view their application statuses.
11. FR11: The system shall check posting capacity and place additional applicants on a waitlist when capacity has been reached, according to the implemented workflow.
12. FR12: The system shall allow companies to view applicants and manage application statuses.
13. FR13: The system shall support interview scheduling and relevant notifications where configured.

Non-Functional Requirements

Performance

The application should respond in a timely fashion for the expected local development scenario. Actual assertions about response-time should only be made after measuring performance.

Security

Authentication should be performed using the backend's role aware routes and token based sessions. Passwords and other secrets should be handled by the configured environment and never included in screenshots, source-control commits, or the final paper.

Usability

The interfaces should be different for student and company users, report application statuses in a human-readable manner, and show what percentage of matches is required and what skills are needed.

Compatibility

The configuration described in the document is based on a modern browser, Node.js with npm, MySQL, and Python, as stated in the instructions. All services, including the frontend, backend, machine-learning service, and database, should be properly configured and operational to ensure the application’s workflow.

Algorithm Formula

A. Resume Skill Extraction

Step 1: extract text from uploaded PDF using a pdf text-extraction library. Step 2: convert it to lowercase. Step 3: match against each configured skill alias (using word-boundary-aware matching). Step 4: map each found alias to its canonical skill name and deduplicate. Step 5: store found skills and estimated proficiency into the DB.

For instance, for resume text “Built a project using JS and React.js,” one may have a controlled vocabulary that maps “JS” to JavaScript and “React.js” to React. However, finding these terms in the extracted text does not establish that the student is proficient in these skills.

B. Proficiency Estimation

The classifier is trained on TF-IDF features using logistic regression to choose between one label: ‘Beginner’, ‘Intermediate’, or ‘Advanced.’ The implementation maps these levels to their stored numeric levels: ‘Beginner’ = 35, ‘Intermediate’ = 65, ‘Advanced’ = 88. These are values from the project’s internal scoring system, not percentages of a true level of ability.

The training script contains 18 skill names 10 sentence templates 3 proficiency classes = 540 synthetic labelled examples. At 80/20 split, that is 432 examples for training on this task, and 108 for testing. The included training report should be quoted for the actual measured test accuracy; even if it says 100%, this only describes performance on the synthetic test set, and does not prove 100% accuracy on real resumes.

C. Internship Match Score

For a posting with required skills, the implementation first identifies the matched skills. Let R be the set of required skills, M the matched skills, w_i the posting weight for matched skill i, and l_i the student's stored level for skill i.

Coverage = $|M| / |R|$

Strength = $\Sigma(l_i \times w_i) / \Sigma(100 \times w_i)$, for matched skills

Match Score = $\text{round}((0.60 \times \text{Coverage} + 0.40 \times \text{Strength}) \times 100)$

Thus, 60% of the score comes from the proportion of required skills matched, and 40% comes from the weighted proficiency levels of those matched skills. Posting weights influence the strength component. The code returns 0 if the posting has no required skills and returns 12 if there are required skills but no matched skills.

System Presentation (Step by Step Documentation)

7. Start the local MySQL database and confirm the ITern database and application user are configured.
8. Start the Express backend from the backend directory after installing dependencies and configuring its environment file.
9. Start the FastAPI service from the ml directory and confirm it is available on port 8000.
10. Serve the frontend on port 5500 and open http://127.0.0.1:5500 in a browser.
11. Register or sign in as a student, complete email verification if SMTP is configured, and open the student profile.
12. Add profile details and upload a text-readable PDF resume. Review the extracted skills and proficiency labels as estimates that may need correction.
13. Open the matches page and review postings, required skills, and the calculated match percentage.
14. Apply to an internship and check the application status; if the posting has reached capacity, verify the waitlist behavior.
15. Sign in through the company portal, create or manage an internship posting, and review applicants and their match scores.
16. Use the company-side controls to update application status or schedule an interview, then verify the corresponding student-facing information.

References

Bhatia, V., Rawat, P., Kumar, A., & Shah, R. R. (2019). End-to-end resume parsing and finding candidates for a job description using BERT. arXiv. https://arxiv.org/abs/1910.03089

Bogen, M., & Rieke, A. (2018). Help wanted: An examination of hiring algorithms, equity, and bias. Upturn.

de Ruijt, C., & Bhulai, S. (2021). Job recommender systems: A review. arXiv. https://arxiv.org/abs/2111.13576

Deshmukh, A., & Raut, A. (2025). Applying BERT-based NLP for automated resume screening and candidate ranking. Annals of Data Science, 12(2), 591–603. https://doi.org/10.1007/s40745-024-00524-5

Freire, M. N., & de Castro, L. N. (2021). e-Recruitment recommender systems: A systematic review. Knowledge and Information Systems, 63, 1–20. https://doi.org/10.1007/s10115-020-01522-8

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied logistic regression (3rd ed.). Wiley.

ITern project source code. (2026). skill_matcher.py, train_proficiency_model.py, export_pure_model.py, and models/training_report.txt [Unpublished course project files].

Kumar, D., Grosz, T., Rekabsaz, N., Greif, E., & Schedl, M. (2023). Fairness of recommender systems in the recruitment domain: An analysis from technical and legal perspectives. Frontiers in Big Data, 6, Article 1245198. https://doi.org/10.3389/fdata.2023.1245198

Lavi, D., Medentsiy, V., & Graus, D. (2021). conSultantBERT: Fine-tuned Siamese Sentence-BERT for matching jobs and job seekers. Proceedings of the RecSys in HR 2021 Workshop. https://arxiv.org/abs/2109.06501

Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), Recommender systems handbook (pp. 73–105). Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Poncio, F. P. (2024). Navigating techniques in job recommender systems on internship profile matching: A systematic review. Journal of Research in Innovative Teaching & Learning, 17(2), 352–367.

Qin, C., Zhu, H., Xu, T., Zhu, C., Jiang, L., Chen, E., & Xiong, H. (2018). Enhancing person-job fit for talent recruitment: An ability-aware neural

network approach. Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval.

Raghavan, M., Barocas, S., Kleinberg, J., & Levy, K. (2020). Mitigating bias in algorithmic hiring: Evaluating claims and practices. Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, 469–481.

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). Recommender systems handbook (2nd ed.). Springer.

Robertson, S. (2004). Understanding inverse document frequency: On theoretical arguments for IDF. Journal of Documentation, 60(5), 503–520.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

Sarawagi, S. (2008). Information extraction. Foundations and Trends in Databases, 1(3), 261–377.

Sebastiani, F. (2002). Machine learning in automated text categorization. ACM Computing Surveys, 34(1), 1–47.

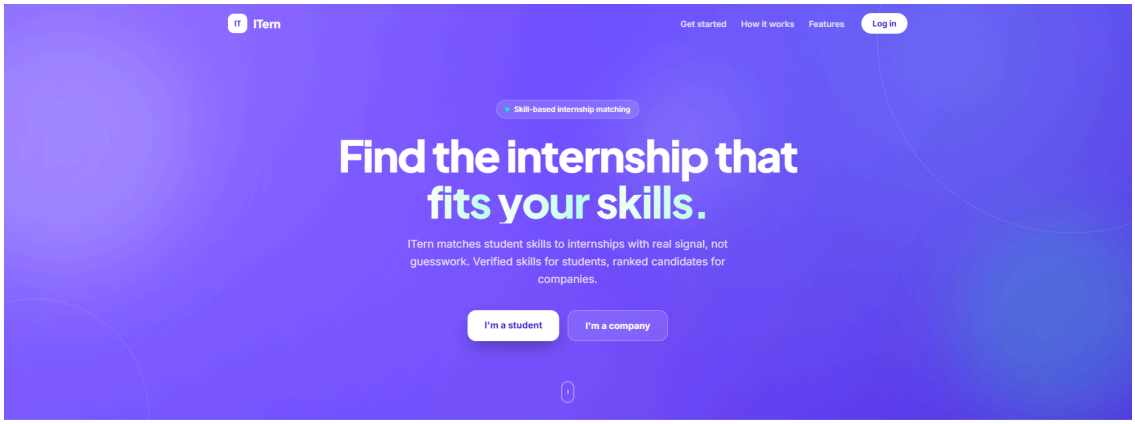
Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.

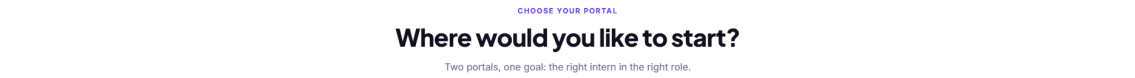
Zhu, C., Zhu, H., Xiong, H., Ma, C., Xie, F., Ding, P., & Li, P. (2018). Person-job fit: Adapting the right talent for the right job with joint representation learning. arXiv. https://arxiv.org/abs/1810.04040

Appendices

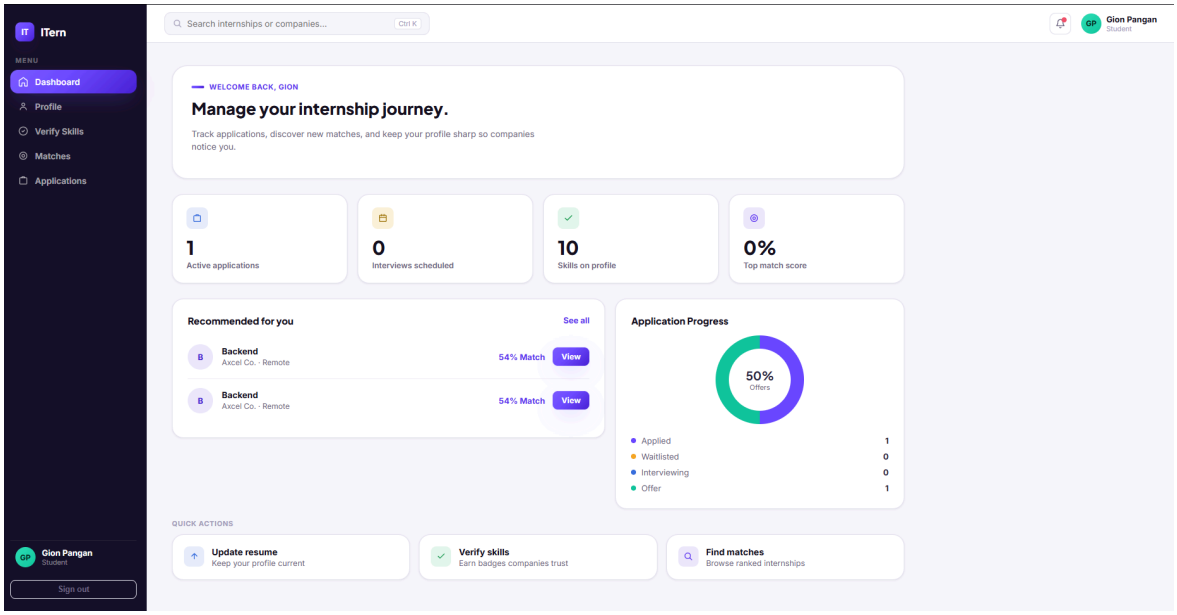
Appendix A: Screenshots

STUDENT VIEW

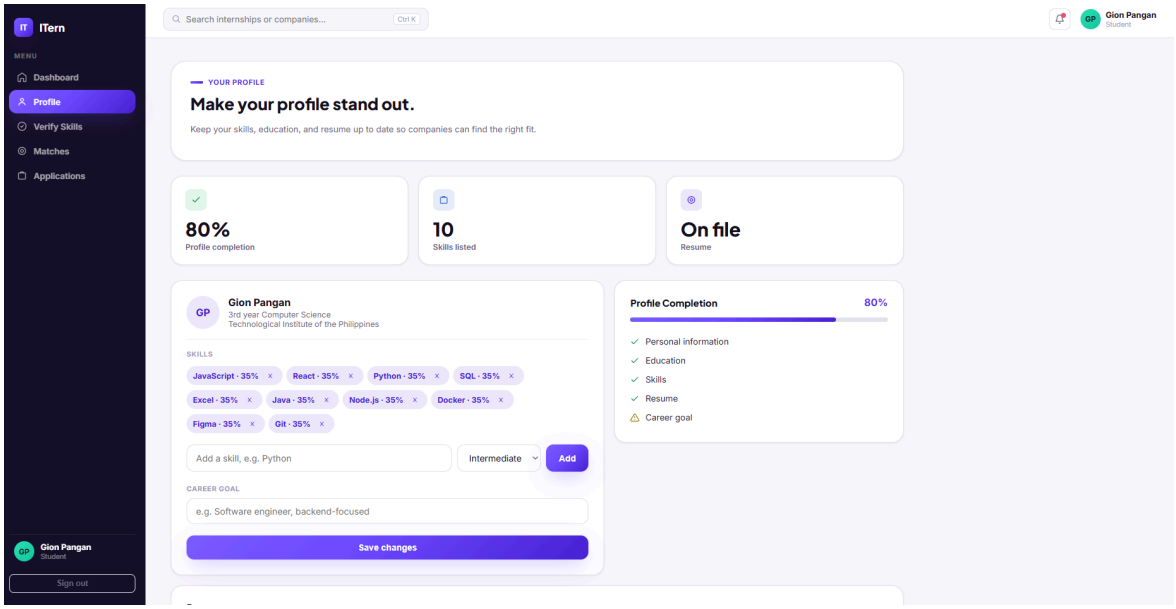




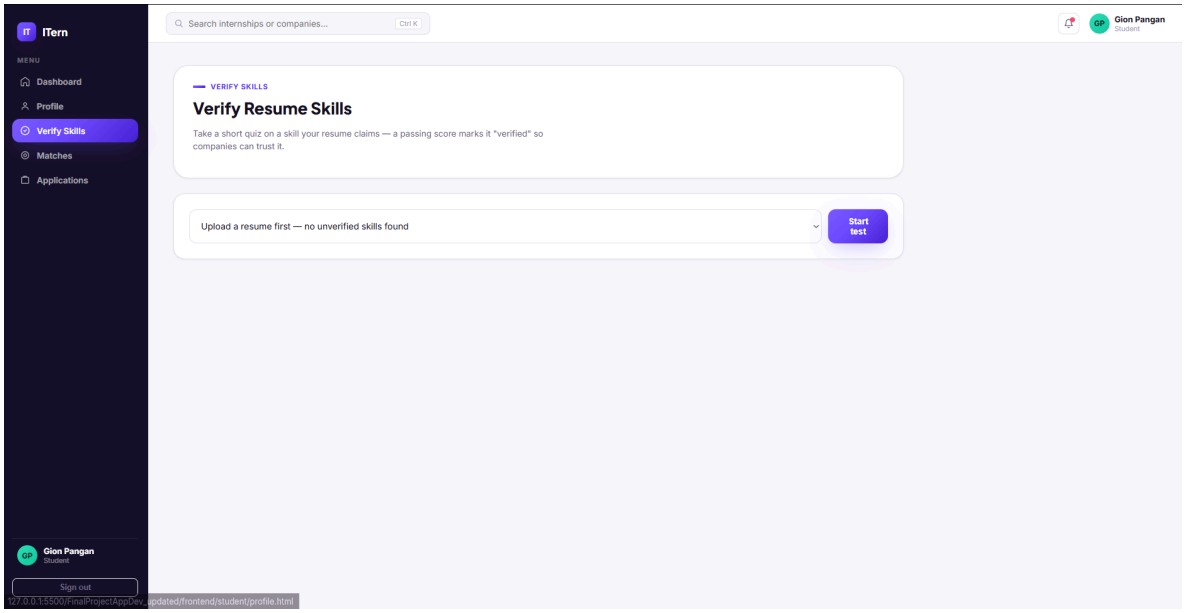
Dashboard



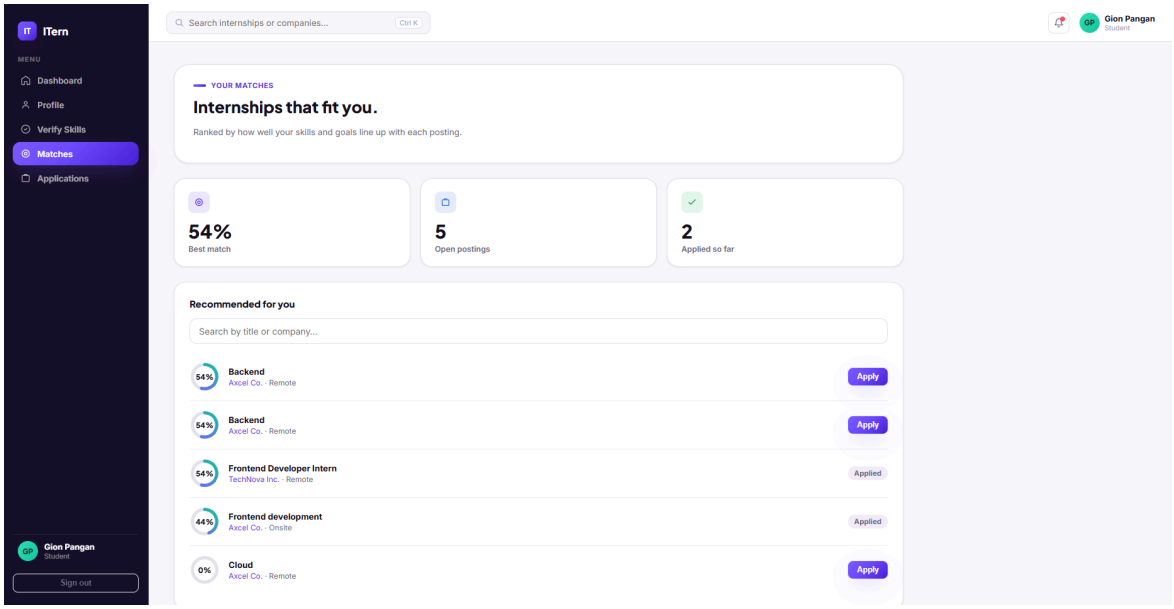
Profile



Verification of skills

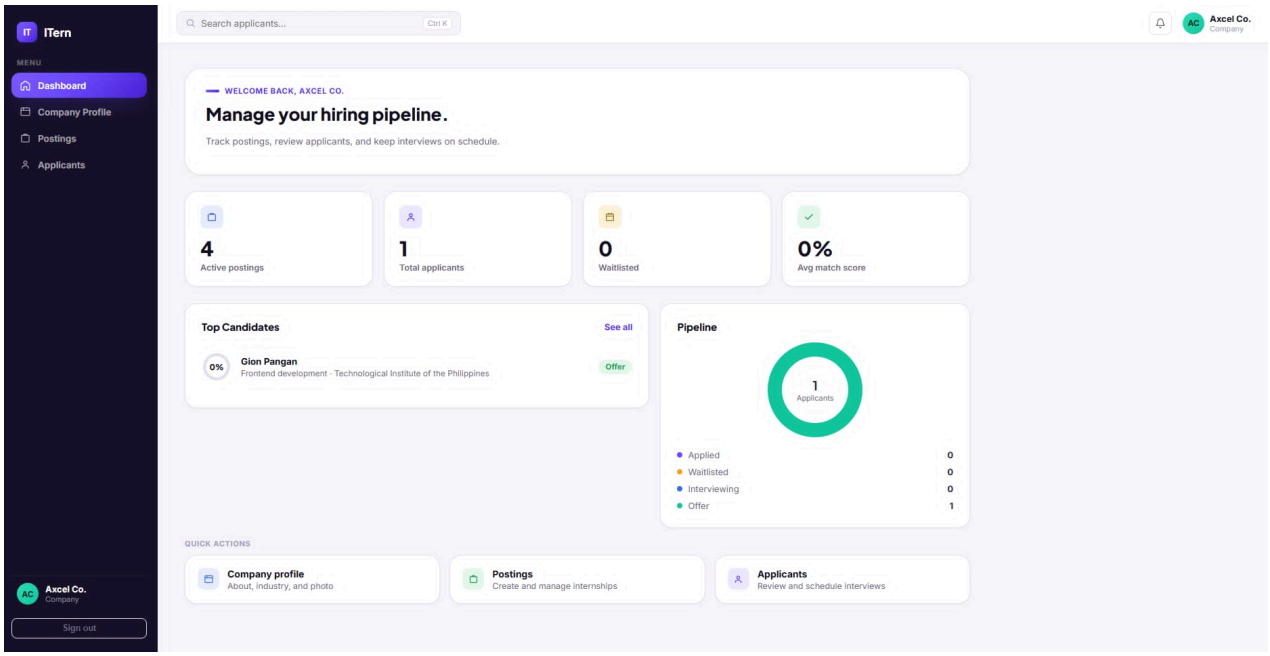


Matches

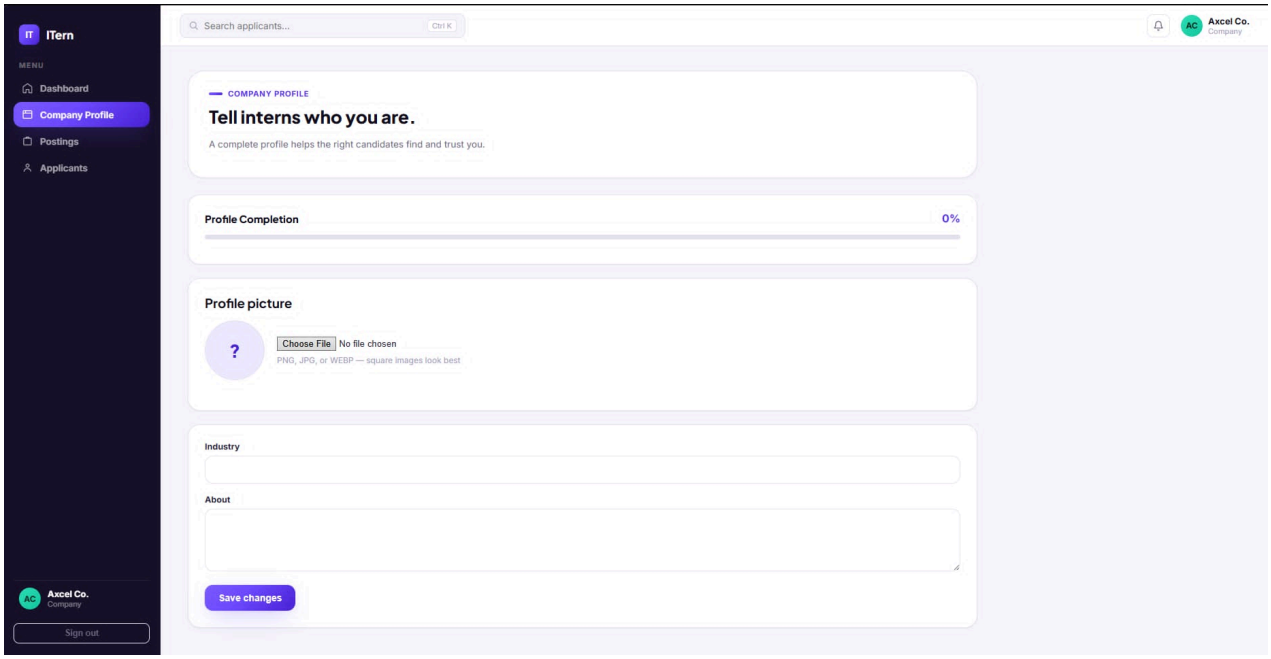


COMPANY VIEW

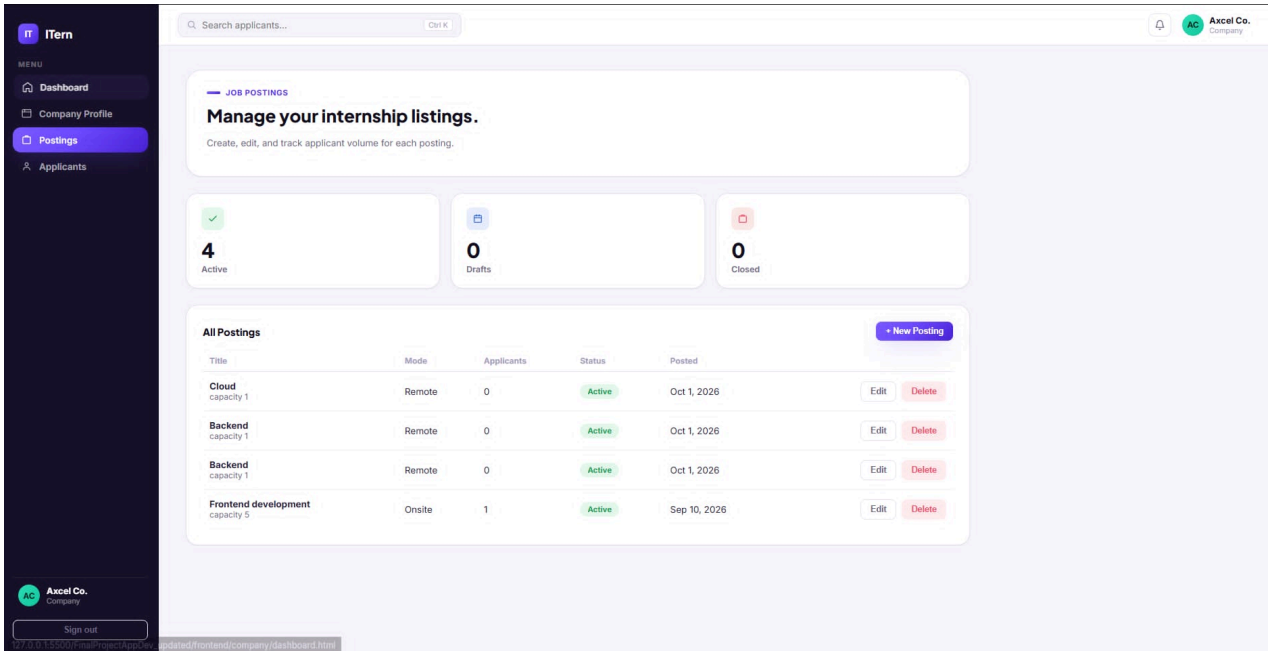
Company Dashboard

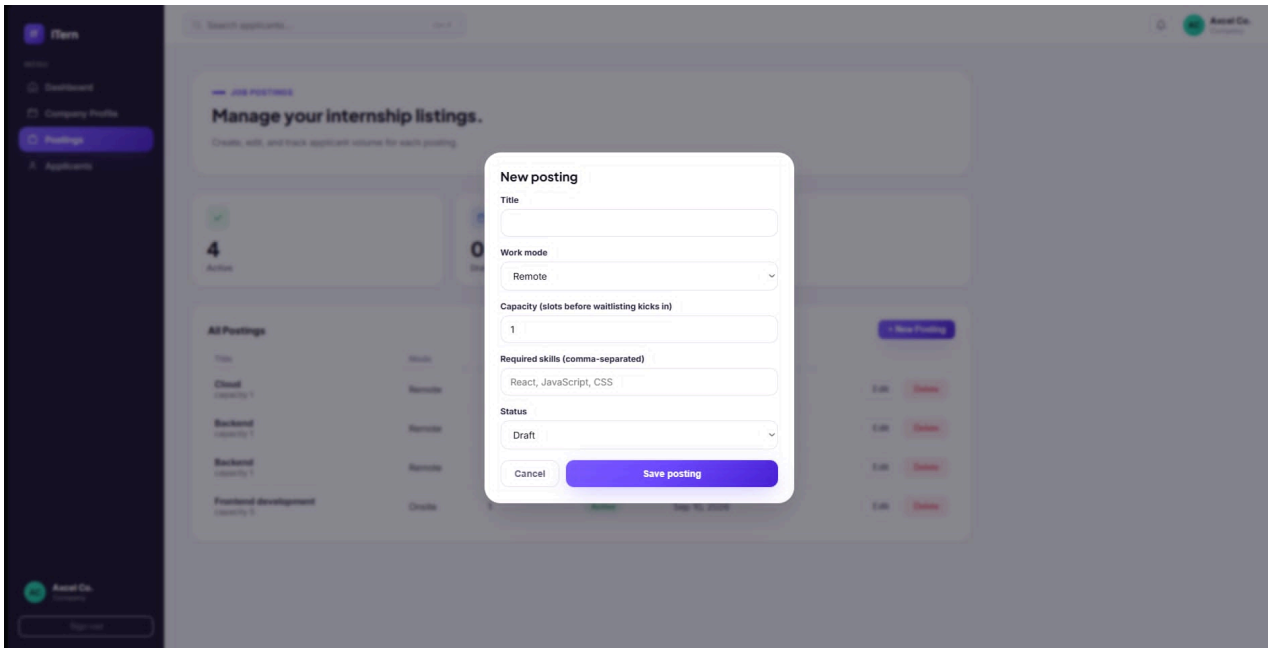


Company Profile

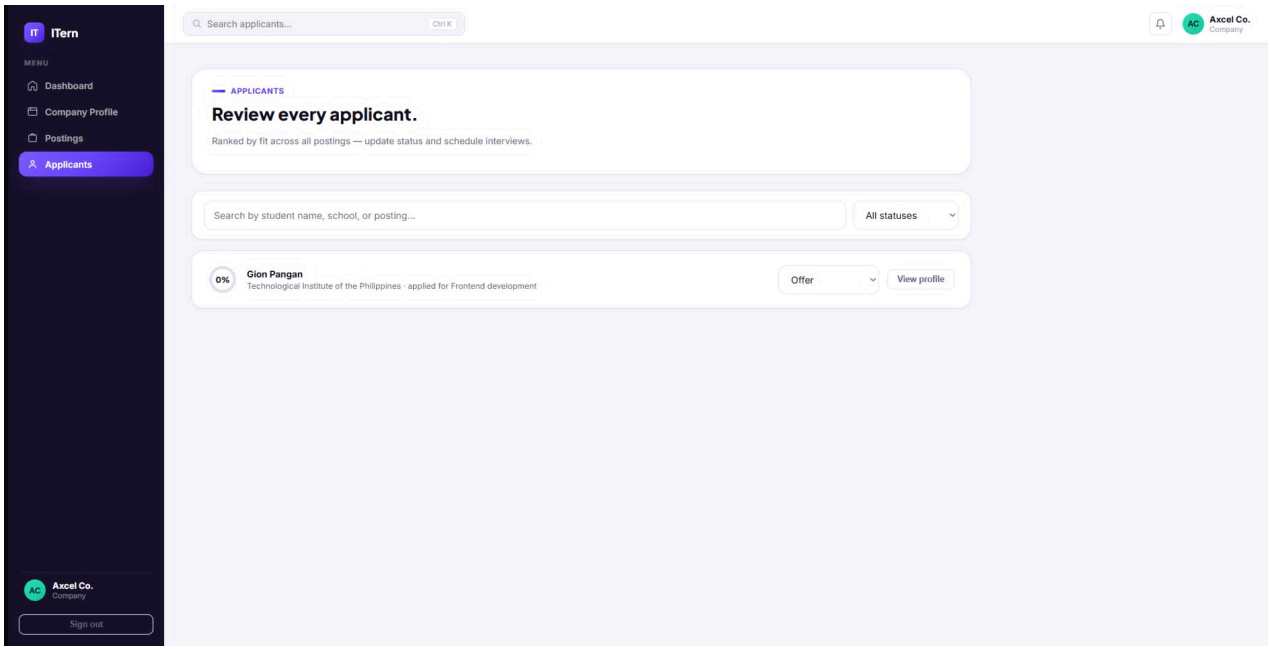


Company Postings





Company Applicants



Resume Template

Juan Dela Cruz

juan.delacruz@example.edu | +63 900 000 0000 | linkedin.com/in/juandelacruz | github.com/juandelacruz

EDUCATION

De La Salle University — BS Computer Science

Expected graduation: May 2027 | GPA: 3.7 / 4.0

SKILLS

Programming Languages: Python, JavaScript, Java, SQL

Frameworks & Tools: React, Node.js, Express, Git, Docker

Other: Figma, Excel, Public Speaking

Tip: list each skill by its common name (e.g. "React" not "ReactJS framework") — this is what gets matched against internship postings.

EXPERIENCE

Freelance Web Developer — Self-employed

June 2024 – Present

- Built and maintained websites for 3 small businesses using React and Node.js
- Improved page load speed by 40% through image optimization and caching

Peer Tutor, Intro to Programming — DLSU CS Department

August 2023 – May 2024

- Tutored 15+ students weekly in Python fundamentals and debugging

PROJECTS

Campus Marketplace App — React, Node.js, MySQL

- Built a buy/sell platform for students with search, messaging, and listings

Weather Dashboard — JavaScript, REST APIs

- Consumed a public weather API to display 5-day forecasts with charts

CERTIFICATIONS (OPTIONAL)

- freeCodeCamp — Responsive Web Design (2024)

This is a starting template — replace every line with your own information, delete sections that don't apply, and save as PDF before uploading to your ITern profile.

Appendix B: Evidence for the Match Percentage

This appendix will provide the supporting evidence for the calculated match percentage of 70%. It is important that the table includes the demonstrated skills or qualifications required by the job, the corresponding weight of each demonstrated skill in the job posting, whether the student possesses the skill or qualification, each student proficiency level for the skills that they possess, the coverage of the skill set that matches the criteria demonstrated in the job posting, and the calculation that was used to determine the weighted-level strengths and ultimately the final match percentage.

Appendix C: Training Data Disclosure

The proficiency model has been trained on synthetic sentences created using particular templates. This means that training examples were not mined from any source, including online resumes, school format resumes, and a resume database.